Project Baseline

Project Baseline

Metrics

Metric	Value	Status
Performance	**31 / 100**	Poor
Accessibility	**79 / 100**	Fair
Best Practices	**54 / 100**	Poor
SEO	**85 / 100**	Good
Largest Contentful Paint (LCP)	**3.6 s**	Needs Improvement
Interaction to Next Paint (INP)	**155 ms**	Good
Cumulative Layout Shift (CLS)	**0.03**	Good
Overall Assessment	**Failed**	❌
First Contentful Paint (FCP)	**1.6 s**	Good

Largest Contentful Paint (LCP)	**5.1 s**	Poor
Speed Index	**10.4 s**	Poor
Total Blocking Time (TBT)	**3,830 ms**	Poor
Cumulative Layout Shift (CLS)	**0.064**	Good

Networking Statistics

Hard Reload

Metric	Value
Requests	**736**
Data Transferred	**9.9 MB**
Total Resource Size	**36.6 MB**
Finish Time	**7.13 s**
DOMContentLoaded	**2.52 s**

Soft Reload

Metric	Value
Requests	**679**
Data Transferred	**1.0 MB**

Total Resource Size	**36.4 MB**
Finish Time	**6.60 s**
DOMContentLoaded	**2.97 s**
Load	**6.21 s**

Cache Savings

Hard Reload:

9.9 MB

Soft Reload:

1.0 MB

Data Saved:

8.9 MB

Reduction:

≈90%

Resource Breakdown

Resource	Transferred	Resource Size
JavaScript	89.1 kB	3.42 MB
Images	640 kB	21.9 MB
CSS	0 kB	2.13 MB

Corrective Findings

Finding 1 – Very Large Network Payload

User Impact

The homepage downloads a very large amount of content, including images, advertisements, videos, and third-party resources. This increases loading time, particularly for users on slower internet connections or mobile networks.

Metrics Affected

- Performance Score
- Largest Contentful Paint (LCP)
- First Contentful Paint (FCP)
- Speed Index

Most Likely Cause

The homepage contains approximately **36.6 MB** of resources, with images making up nearly **22 MB** of the total page size.

Likely Solution

Compress images further, lazy-load media below the fold, reduce unnecessary downloads during the initial page load, and remove unused assets where possible.

Finding 2 – Excessive JavaScript Blocking

User Impact

Although content appears on screen, the page remains busy processing JavaScript, making interactions feel delayed.

Metrics Affected

- Total Blocking Time (TBT)
- Performance Score

Most Likely Cause

More than **3.4 MB** of JavaScript is loaded for advertisements, analytics, comments, videos, and other interactive features, creating long tasks on the browser's main thread.

Likely Solution

Reduce unused JavaScript, split large bundles into smaller chunks, and defer non-essential scripts until after the main content has loaded.

Finding 3 – Extremely High Number of Network Requests

User Impact

The browser must establish and manage hundreds of individual requests before the page fully loads, increasing overall loading time and network overhead.

Metrics Affected

- Finish Time
- Largest Contentful Paint (LCP)
- Performance Score

Most Likely Cause

The homepage loads **736 requests**, including news articles, advertisements, analytics services, fonts, scripts, and multimedia content.

Likely Solution

Reduce unnecessary requests, combine smaller assets where practical, and lazy-load resources that are not immediately visible.

Finding 4 – Images Dominate the Download Size

User Impact

Large images consume most of the downloaded data, increasing loading time and bandwidth usage, particularly on mobile devices.

Metrics Affected

- Largest Contentful Paint (LCP)
- Speed Index
- Finish Time

Most Likely Cause

Images account for approximately **21.9 MB** of the **36.6 MB** total page resources.

Likely Solution

Resize oversized images, compress them further, and use responsive images in modern formats such as WebP or AVIF.

Finding 5 – Heavy Dependence on Third-Party Resources

User Impact

Performance depends on external advertising, analytics, and tracking services. If any of these services respond slowly, users experience longer loading times.

Metrics Affected

- Performance Score
- Largest Contentful Paint (LCP)
- Total Blocking Time (TBT)
- Speed Index

Most Likely Cause

The page loads numerous third-party advertising and analytics scripts during the initial page load.

Likely Solution

Delay loading non-essential third-party services until after the primary content is displayed and remove services that provide little value.

Good Findings

Effective HTTP Compression

The website reduces transferred data from **36.6 MB** of resources to **9.9 MB** during the initial load, achieving approximately **73% compression**. This significantly lowers bandwidth usage without affecting content quality.

Excellent Browser Caching

A normal page refresh transfers only **1.0 MB** compared with **9.9 MB** during the initial visit, reducing downloaded data by approximately **90%**. CSS is served entirely from cache, making repeat visits much more efficient.

Overall Conclusion

AP News demonstrates good caching and compression practices, which substantially reduce bandwidth usage for returning visitors. However, its homepage is extremely resource-intensive, containing hundreds of network requests, large images, numerous third-party services, and several megabytes of JavaScript. These factors contribute to a low Performance score of **31**, a **5.1-second Largest Contentful Paint**, and a **3,830 ms Total Blocking Time**. Improving image optimization, reducing JavaScript execution, minimizing third-party resources, and lowering the total number of requests would significantly improve loading speed and overall user experience.

Mobile Metrics

Mobile Metrics

Metric	Value	Rating
Performance Score	**29 / 100**	🔴 Poor
Accessibility	**76 / 100**	🟠 Needs Improvement
Best Practices	**77 / 100**	🟠 Needs Improvement
SEO	**85 / 100**	🟡 Good
First Contentful Paint (FCP)	**6.9 s**	🔴 Poor
Largest Contentful Paint (LCP)	**40.0 s**	🔴 Poor
Total Blocking Time (TBT)	**1,540 ms**	🔴 Poor
Speed Index	**16.8 s**	🔴 Poor
Cumulative Layout Shift (CLS)	**0.027**	🟢 Good

Finding 1 – Extremely Slow Largest Contentful Paint

Mobile users wait a very long time before the main content of the page becomes visible. Many users may leave the page before the largest image or article is fully loaded, especially on slower mobile networks.

Metrics Affected

- Largest Contentful Paint (40.0 s)
- First Contentful Paint (6.9 s)

- Speed Index (16.8 s)
- Overall Performance Score (29)

Most Likely Cause

The report shows several render-blocking JavaScript and CSS files delaying the initial rendering of the page. It also identifies very large images that are not optimized for mobile devices, with an estimated image delivery saving of about **2 MB**.

Recommended Solution

Reduce render-blocking resources by deferring non-critical JavaScript and inlining critical CSS. Optimize and compress large images, serve responsive image sizes, and lazy-load images that are not immediately visible.

Finding 2 – Inefficient Caching and Heavy Third-Party Resources

Returning visitors download many resources again instead of using cached versions, resulting in slower page loads and unnecessary network usage on mobile devices.

Metrics Affected

- Largest Contentful Paint (LCP)
- First Contentful Paint (FCP)
- Overall Performance Score

Most Likely Cause

The report recommends using longer cache lifetimes, estimating potential savings of approximately **1,628 KiB**. Many third-party resources, including advertising, analytics, and tracking scripts, use relatively short cache lifetimes, causing them to be downloaded frequently.

Recommended Solution

Increase cache lifetimes for static resources where possible, reduce unnecessary third-party scripts, and load advertising and analytics asynchronously so they do not delay the main page content.

Build Analysis

JavaScript

How is the JavaScript bundled?

The website uses **multiple JavaScript bundles (chunk-based code splitting)** rather than a single large bundle. JavaScript is divided into many separate files, including both application bundles and numerous third-party scripts. Bundle sizes generally range from approximately **50 kB to 460 kB**, indicating that the application has been split into multiple smaller chunks rather than one monolithic bundle.

Is this the right decision?

Yes, Using multiple JavaScript bundles is considered good practice because it improves browser caching, allows only the required code to be downloaded initially, and avoids shipping one excessively large JavaScript file. Modern browsers can efficiently download multiple bundles in parallel using HTTP/2 and HTTP/3.

However, the benefits are partially offset by the large number of third-party scripts that are also loaded during the initial page load.

Is there unused JavaScript?

Yes, The Coverage report indicates that a significant portion of the downloaded JavaScript is unused during the initial page load.

Examples include:

File	Unused JavaScript
assets.apnews.com/.../All.min...	**82.5%**
live.prime.tech prebidVid	**64.5%**
CookieLaw Banner SDK	**72.4%**
reCAPTCHA	**66.1%**

Live Video Player	**91.1%**
Bounce Exchange	**83.8%**

Overall, the Coverage report shows:

- **8.1 MB used**
- **10.1 MB unused**

meaning that approximately **55% of the downloaded JavaScript is unused during the initial page load.**

Although some of this code may be required after user interaction, the results indicate there is considerable opportunity to further optimize the JavaScript delivered initially.

Analyze the bundles. Are they good?

Overall, the JavaScript bundling strategy follows modern best practices by splitting the application into multiple bundles rather than relying on one large file.

Positive observations include:

- JavaScript is divided into many smaller bundles.
- HTTP/2 and HTTP/3 are used for parallel downloads.
- Individual application bundles remain reasonably sized.

However, several opportunities for improvement were identified:

- A large number of third-party JavaScript libraries are loaded during the initial page load.
- Coverage analysis shows that many bundles contain substantial amounts of unused JavaScript.
- Numerous advertising, analytics, tracking, consent management, and video-related scripts increase the amount of JavaScript downloaded and executed before the page becomes fully interactive.

Are source maps available? Should they be?

No public source maps were detected.

This is considered good practice for a production website because it prevents exposure of the original application source code while slightly reducing production asset size.

CSS

How is the CSS bundled?

The website uses **multiple CSS bundles (chunk-based CSS splitting)** rather than a single stylesheet. The CSS is divided into numerous separate files, including both application-specific stylesheets and third-party CSS resources. This indicates that the website uses a modern bundling strategy that splits CSS into smaller chunks instead of delivering one large stylesheet.

Is this the right decision?

Yes, Using multiple CSS bundles is generally considered good practice because it:

- reduces the size of individual stylesheets,
- improves browser caching,
- allows different pages or components to load only the styles they require,
- avoids maintaining one excessively large CSS file.

The downloaded CSS files are generally small, with only one larger application stylesheet and numerous smaller supporting stylesheets.

Is there unused CSS?

Yes, The Coverage report shows that a significant amount of downloaded CSS is unused during the initial page load.

Examples include:

File	Unused CSS
All.min.70c2eda7933b1b601140d901727ba769.gz.css	**88.5%**

Viafoura CSS	**97.0%**
Viafoura CSS (chunk)	**99.9%**
Google Fonts (Roboto / Merriweather)	**100%**
Google Fonts (Poppins)	**100%**
Google Fonts (Assistant)	**100%**

These results indicate that a considerable amount of CSS is downloaded before it is required by the page. While some unused CSS is expected because certain components and third-party features are loaded for later interactions, the consistently high percentages suggest there is an opportunity to further optimize CSS delivery by loading styles only when needed.

Images

Are the images shipped in multiple sizes/formats?

Yes, The website serves images in multiple modern formats, including **WebP**, **AVIF**, and **JPEG**, as shown in the Network panel. The presence of multiple image formats allows browsers to download the most appropriate version depending on browser support.

Several image URLs also contain resizing parameters, for example:

- `width=474&height=315`
- `fit=cover&height=400&width=...`

These parameters indicate that images are dynamically resized before being delivered rather than always serving a single fixed-size image. This demonstrates that the website provides multiple image sizes optimized for different layouts and devices.

Is this the right decision?

Yes, This is considered a good optimization strategy because:

- Modern image formats such as **WebP** and **AVIF** provide significantly smaller file sizes than traditional JPEG images while maintaining similar visual quality.
- Images are dynamically resized based on the requested dimensions, reducing unnecessary bandwidth usage.
- Different image formats can be served depending on browser capabilities, improving compatibility and performance.
- Delivering appropriately sized images helps reduce download time and improves page loading performance, especially on mobile devices.

Overall, the image delivery strategy follows modern web performance best practices.

Is the full-resolution file exposed?

No evidence suggests that the original full-resolution image is exposed.

The analyzed images are served through optimized image URLs that include resizing and optimization parameters, rather than exposing a direct original asset. In addition, many images are delivered in optimized formats such as **WebP** and **AVIF**, while resized versions are requested through URLs containing width and height parameters.

The largest image captured in the Network panel is served directly as a JPEG from a dedicated asset domain (`assets.riverdrop.com`) and does not expose any indication of an original higher-resolution version. Based on the available network data, there is no evidence that the original full-resolution images are publicly exposed to users.

Third-Party Resources

1. What third-party tools are used?

The website uses a large number of third-party services for analytics, advertising, user engagement, video delivery, notifications, and consent management.

Identified third-party services include:

Third-Party Tool	Purpose
Google Tag Manager (GTM)	Loads and manages marketing and analytics tags.

Google Analytics (GA4)	Website analytics and visitor tracking.
Google reCAPTCHA	Bot protection and spam prevention.
Google Publisher Ads / DoubleClick (GPT)	Advertisement delivery.
Amazon Publisher Services (apstag.js)	Header bidding for advertisements.
Primis	Video advertising and video content delivery.
Viafoura	User comments and community engagement.
OneSignal	Push notifications.
Bounce Exchange	Marketing and visitor engagement.
CookieLaw (CookieYes)	Cookie consent management.
BlueConic	Customer data platform and personalization.
Dianomi	Native advertising.
NTV.io	Video streaming and advertising.
Sail Horizon	Advertising and tracking services.
RapidEdge Metrics	Performance and analytics monitoring.

---

2. How are they loaded?

Most third-party resources are loaded during the **initial page load** rather than being delayed or lazy-loaded.

Evidence from the Network waterfall shows that many third-party JavaScript files begin downloading within the first few seconds alongside the website's own resources. These include:

- Google Tag Manager
- Google Analytics
- Google reCAPTCHA
- Amazon Publisher Services
- Primis
- Viafoura
- Bounce Exchange
- OneSignal
- CookieLaw
- Dianomi

Loading numerous third-party services early increases the amount of JavaScript that must be downloaded and executed before the page becomes fully interactive.

---

3. How much is each one impacting page load or other performance metrics?

The third-party services contribute significantly to:

- the total number of network requests,
- JavaScript download size,
- main-thread JavaScript execution,
- Total Blocking Time (TBT),
- overall page loading time.

The Coverage report also shows that many third-party scripts contain a substantial amount of unused JavaScript during the initial page load.

Examples include:

Third-Party Script	**Approximate Unused JavaScript**
NTV.io	78%
Google reCAPTCHA	66%
Primis	91%
CookieLaw	72%
Bounce Exchange	67–80%
Viafoura	48–91%
Google Tag Manager	~40%
Amazon APS	~41%
OneSignal	~71%
BlueConic	~75%

Although these services provide useful functionality, a considerable portion of their JavaScript is not immediately required during the initial rendering of the homepage.

4. Do any seem inappropriate or unnecessary?

No individual third-party service appears inherently inappropriate for a large news website. Advertising platforms, analytics, comment systems, video services, cookie consent management, and push notifications are all common components of modern online news platforms.

However, the website relies on a very large number of third-party services that are loaded during the initial page load. Several of these scripts contain high percentages of unused JavaScript during the initial rendering, indicating that some could potentially be deferred or lazy-loaded. Reducing the amount of third-party JavaScript executed during startup could improve loading performance, reduce Total Blocking Time, and enhance the overall user experience without removing essential functionality.

Corrective Findings

Finding 1 – Large Amount of Unused JavaScript

The Coverage report shows that approximately **55% of the downloaded JavaScript (10.1 MB out of 18.2 MB)** is unused during the initial page load. Several JavaScript bundles and third-party scripts contain between **64% and 91% unused code**. Reducing unused JavaScript through additional code splitting, deferred loading, or lazy loading could reduce JavaScript execution time and improve page responsiveness.

Finding 2 – Large Amount of Unused CSS

The Coverage report indicates that many CSS files contain extremely high percentages of unused styles during the initial page load. Several stylesheets, including application CSS and third-party resources such as Viafoura and Google Fonts, contain between **88% and 100% unused CSS**. Loading only the styles required for the current page could reduce unnecessary downloads and improve rendering efficiency.

Finding 3 – Large Number of Third-Party Scripts Loaded During Initial Page Load

The website loads numerous third-party services, including Google Tag Manager, Google Analytics, Google Publisher Ads, Amazon Publisher Services, Primis, Viafoura, CookieLaw, OneSignal, BlueConic, and Bounce Exchange during the initial page load. Coverage analysis shows that many of these scripts contain substantial amounts of unused JavaScript. Deferring or lazy-loading non-essential third-party resources could reduce network requests, decrease Total Blocking Time, and improve overall loading performance without affecting core website functionality.